CSE 276A — HW1 Grading Overview (Skim)

Student: Rui Zhang

Robot/software: Waveshare Wave Rover, Rubik Pi, Ubuntu 24.04, ROS 2 Jazzy

Video of example run: https://youtu.be/eSfmiSPTKbU?feature=shared (https://youtu.be/eSfmiSPTKbU?feature=shared)

Detailed write-up: `README_HW1.md` (README_HW1.md)

Outcome: The first two waypoint transitions worked, and an initial full set of six motion legs ran with substantial drift. The subsequent compensated attempt **did not accurately finish the route**: after `(2,2)`, its direction was broadly toward `(1,2)` rather than the intended `(1,1)`. The complete-route final pose error was **not measured**.

> *This summary follows the explicit homework requirements: model, algorithm/control, Python/ROS 2 implementation, calibration, robot experiment/video, and performance discussion. The assignment handout does not provide category point weights, so none are claimed here.*

1. Kinematic model and algorithm

We approximate the four-wheel skid-steer rover with differential-drive kinematics:

[v=\frac{v_R+v_L}{2},\qquad \omega=\frac,\qquad \dot x=v\cos\theta,\quad \dot y=v\sin\theta,\quad \dot\theta=\omega.]

Because ROS left/right motor commands are **normalized inputs**, not encoder-measured wheel speeds, we empirically calibrate motor **duration → translation/rotation** rather than assume known wheel speed or a slip-free track width. The seven `(x,y,θ)` waypoints are read from `config/waypoints.txt`, with x/y scaled by **0.8 m per coordinate unit** and θ retained in radians. The controller starts from a manually aligned `(0,0,0)` pose.

For each target: calculate `atan2(Δy,Δx)` and Euclidean distance; apply a signed, wrapped **initial turn → straight drive → final turn to the requested θ**. ROS 2 publishes `std_msgs/msg/Float32MultiArray([left,right])` on `/motor_commands` at **25 Hz**; the course `motor_control` bridge sends the commands to the STM32 over USB serial. Motions end with repeated zero commands. The method remains **open loop**: the next segment uses predicted, not observed, pose.

A later experimental planner optionally modeled unintended center translation during rotations:

[\Delta p_=(I-R(\delta))c]

with separate empirical effective pivots for left/right turns. It adjusts planned inter-turn translations, **not** actual pose or heading in real time. This model was attempted but **did not solve the whole-route drift**.

2. Calibration and evidence

69 standalone calibration trials are preserved in `calibration/trials.csv`. **Eight measured action-level route checkpoints** are in `calibration/route_checkpoints.csv`; these are additional diagnostic measurements, **not 8 more calibration trials**.

Calibration experiment	Measured result
Forward `[0.20,0.20]`, 3.0 s (n=5)	0.784 m mean forward-axis movement, about 7.82 cm rightward drift
Corrected forward `[0.130,0.200]`, 4.0 s (n=2)	**0.798 m** mean forward-axis movement, about **0.65 cm** rightward drift
Earlier left / right turn at 0.95 s	88.0° left / 99.3° right means (three repetitions each)
Later painter-tape condition, left 0.70 s (n=2)	Mean **89.5°**
Later painter-tape condition, right 0.90 s (n=2)	Mean **86.5°**
Later condition, right 2.20 s (n=2)	182° and 188°

Mechanical condition changed during calibration: the right-rear wheel became intermittently ineffective, and painter's tape was applied near it. Earlier and later turning measurements therefore represent different conditions; the tape was not verified as a safe mechanical fix. The controller uses empirically interpolated left/right turn durations, including different 90° and 180° right-turn behavior.

In the eight-checkpoint diagnostic, four nominal 0.8 m straight commands achieved approximately **0.750, 0.734, 0.663, 0.650 m** in the robot's initial forward direction for each motion. A provisional speed estimate was reduced from **0.19949 to 0.17441 m/s**, and separate left/right effective-pivot shifts were fitted. **Those changes were not validated as a successful tracking correction.**

3. Robot test and observed performance

- **Initial 1- and 2-leg tests:** Both executed; a left turn produced roughly **5–15 cm unintended center translation**.
- **First six-leg run:** The motion sequence was executed and its overall path was recognizable, but deviations grew. At the `(2,2)` approach, the rover was visually estimated around `(2,1.7)` before the final turn (about **0.24 m** world-y shortfall after scaling; qualitative estimate).
- **Separate measured four-leg diagnostic:** After the final orientation turn at original waypoint `(2,2)`, the measured pose was **`(1.71,1.30,-88°)`**, versus target **`(1.60,1.60,-90°)`**. Checkpoint position error was about **0.320 m**; this is **not** the final error of all six legs.
- **Later compensated attempt:** Despite modeled turn-center shifts and retuned forward duration, yaw drift accumulated. From around `(2,2)`, the next movement headed toward **`(1,2)` rather than `(1,1)`**. Full-route tracking remained inaccurate, with **no measured final endpoint**.

Video: https://youtu.be/eSfmiSPTKbU?feature=shared (https://youtu.be/eSfmiSPTKbU?feature=shared) — supplied recording link for the report (the report does not claim that the video independently establishes pose accuracy).

4. Discussion: why it failed and what would improve it

The principal limitation was **accumulating open-loop orientation and position error**. Measured turns translated the chassis center by several centimeters, right/left turn timing differed, and one nominal right turn changed yaw by only about 70°. On carpet with the altered drivetrain, straight travel varied substantially. A small heading error redirects the *entire next forward leg*; position-only compensation still starts each leg from a **predicted**, potentially wrong, heading. The effective-pivot approximation is also imperfect if skidding and wheel behavior vary between turns.

What worked: waypoint parsing/scaling, differential-drive approximation, ROS command streaming, calibrated motion primitives, short autonomous segments, and full route action sequencing. **What did not:** repeatable six-leg pose accuracy. Potential improvements are first to restore a mechanically sound drivetrain, then use heading/position feedback (IMU where suitable, wheel odometry if reliable, or external vision/markers) and shorter closed-loop motion updates. These were not part of the implemented open-loop solution.

5. Code, reproducibility, and submission

- `robot_control/waypoint_controller.py`: waypoint parsing, open-loop planner, optional turn-shift model, ROS publisher, dry run, waypoint/action pauses, workspace checks.
- `robot_control/calibrate_motion.py`: repeatable timed motor-command experiments.
- `config/calibration.json`, `config/waypoints.txt`: empirical parameters and original seven poses.
- `calibration/trials.csv`, `route_checkpoints.csv`, `turn_shift_measurements.csv`, `route_tests.csv`: raw experiments and diagnostics.

Run from the ROS package root on Rubik Pi (with the course's `motor_control` node running in another terminal):

```
source /opt/ros/jazzy/setup.bash
source ~/ros2_ws/install/setup.bash
cd ~/ros2_ws/src/rubikpi_ros2/robot_control
python3 robot_control/waypoint_controller.py --dry-run
```

The full report gives physical-execution options and safety limitations. **Do not rerun physical tests with a binding/stalling wheel or tape interfering with moving parts.** For the Canvas submission, include the **complete course ROS package** (`setup.py`/`package.xml` from the original repository), code and calibration files, the report converted to PDF, and the YouTube URL; exclude build caches and the video binary itself.